In [1]:
```
import numpy as np
import tensorflow as tf

# Input data (X) untuk gerbang logika
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=np.float32)

# Target output untuk masing-masing gerbang
y_and = np.array([[0], [0], [0], [1]], dtype=np.float32)
y_or = np.array([[0], [1], [1], [1]], dtype=np.float32)
y_xor = np.array([[0], [1], [1], [0]], dtype=np.float32)

print("Input (X):\n", X)
print("Target AND (y_and):\n", y_and)
print("Target OR (y_or):\n", y_or)
print("Target XOR (y_xor):\n", y_xor)
```

```
Input (X):
 [[0. 0.]
 [0. 1.]
 [1. 0.]
 [1. 1.]]
Target AND (y_and):
 [[0.]
 [0.]
 [0.]
 [1.]]
Target OR (y_or):
 [[0.]
 [1.]
 [1.]
 [1.]]
Target XOR (y_xor):
 [[0.]
 [1.]
 [1.]
 [0.]]
```

In [2]:
```
from tqdm.notebook import tqdm
import time

# 1. Model untuk Gerbang AND (1 Neuron, Sigmoid)
model_and = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model_and.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

print("--- Training Model AND (YOLO-Style) ---")
epochs = 300
pbar_and = tqdm(total=epochs, desc="Training AND", unit="epoch")

# List untuk menampung seluruh riwayat training
history_and_loss = []
history_and_acc = []

for epoch in range(epochs):
    history_and = model_and.fit(X, y_and, epochs=1, verbose=0)
    current_loss = history_and.history['loss'][0]
    current_acc = history_and.history['accuracy'][0]

    # Simpan riwayat
    history_and_loss.append(current_loss)
```

```
    history_and_acc.append(current_acc)

    pbar_and.set_postfix({
        'loss': f"{current_loss:.4f}",
        'accuracy': f"{current_acc:.4f}"
    })
    pbar_and.update(1)
    time.sleep(0.01)

pbar_and.close()
print("Training Selesai. Akurasi Akhir:", current_acc)
```

```
--- Training Model AND (YOLO-Style) ---
```

Training AND: 100%  300/300 [00:33<00:00, 9.71epoch/s, loss=0.0162, accuracy=1.0000]

```
Training Selesai. Akurasi Akhir: 1.0
```

In [3]:
```
# 2. Model untuk Gerbang OR (1 Neuron, Sigmoid)
model_or = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model_or.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
                 loss='binary_crossentropy',
                 metrics=['accuracy'])

print("--- Training Model OR (YOLO-Style) ---")
epochs = 300
pbar_or = tqdm(total=epochs, desc="Training OR", unit="epoch")

history_or_loss = []
history_or_acc = []

for epoch in range(epochs):
    history_or = model_or.fit(X, y_or, epochs=1, verbose=0)
    current_loss = history_or.history['loss'][0]
    current_acc = history_or.history['accuracy'][0]

    history_or_loss.append(current_loss)
    history_or_acc.append(current_acc)

    pbar_or.set_postfix({
        'loss': f"{current_loss:.4f}",
        'accuracy': f"{current_acc:.4f}"
    })
    pbar_or.update(1)
    time.sleep(0.01)

pbar_or.close()
print("Training Selesai. Akurasi Akhir:", current_acc)
```

```
--- Training Model OR (YOLO-Style) ---
```

Training OR: 100%  300/300 [00:33<00:00, 9.76epoch/s, loss=0.0245, accuracy=1.0000]

```
Training Selesai. Akurasi Akhir: 1.0
```

In [4]:
```
# 3. Model untuk Gerbang XOR
model_xor = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(3, activation='sigmoid'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model_xor.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

print("--- Training Model XOR (YOLO-Style) ---")
```

```
epochs = 300
pbar_xor = tqdm(total=epochs, desc="Training XOR", unit="epoch")

history_xor_loss = []
history_xor_acc = []

for epoch in range(epochs):
    history_xor = model_xor.fit(X, y_xor, epochs=1, verbose=0)
    current_loss = history_xor.history['loss'][0]
    current_acc = history_xor.history['accuracy'][0]

    history_xor_loss.append(current_loss)
    history_xor_acc.append(current_acc)

    pbar_xor.set_postfix({
        'loss': f"{current_loss:.4f}",
        'accuracy': f"{current_acc:.4f}"
    })
    pbar_xor.update(1)
    time.sleep(0.01)

pbar_xor.close()
print("Training Selesai. Akurasi Akhir:", current_acc)
```

```
--- Training Model XOR (YOLO-Style) ---
```

Training XOR: 100%  300/300 [00:36<00:00, 9.20epoch/s, loss=0.0038, accuracy=1.0000]

```
Training Selesai. Akurasi Akhir: 1.0
```

In [5]:

```
# Menampilkan Hasil Prediksi untuk Semua Gerbang
pred_and = model_and.predict(X)
pred_or = model_or.predict(X)
pred_xor = model_xor.predict(X)

print("\nHasil Prediksi Akhir (Nilai Probabilitas & Pembulatan Biner):\n")
for i in range(4):
    print(f"Input: {X[i].tolist()}")
    print(f"  AND -> Prediksi: {pred_and[i][0]:.4f} (Bulat: {int(np.round(pred_and[i][0]))})")
    print(f"  OR  -> Prediksi: {pred_or[i][0]:.4f} (Bulat: {int(np.round(pred_or[i][0]))}) |
    print(f"  XOR -> Prediksi: {pred_xor[i][0]:.4f} (Bulat: {int(np.round(pred_xor[i][0]))})")
    print("-" * 60)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step

Hasil Prediksi Akhir (Nilai Probabilitas & Pembulatan Biner):

Input: [0.0, 0.0]
  AND -> Prediksi: 0.0000 (Bulat: 0) | Target: 0
  OR  -> Prediksi: 0.0536 (Bulat: 0) | Target: 0
  XOR -> Prediksi: 0.0040 (Bulat: 0) | Target: 0
------------------------------------------------------------
Input: [0.0, 1.0]
  AND -> Prediksi: 0.0172 (Bulat: 0) | Target: 0
  OR  -> Prediksi: 0.9804 (Bulat: 1) | Target: 1
  XOR -> Prediksi: 0.9988 (Bulat: 1) | Target: 1
------------------------------------------------------------
Input: [1.0, 0.0]
  AND -> Prediksi: 0.0164 (Bulat: 0) | Target: 0
  OR  -> Prediksi: 0.9775 (Bulat: 1) | Target: 1
  XOR -> Prediksi: 0.9937 (Bulat: 1) | Target: 1
------------------------------------------------------------
Input: [1.0, 1.0]
  AND -> Prediksi: 0.9698 (Bulat: 1) | Target: 1
  OR  -> Prediksi: 1.0000 (Bulat: 1) | Target: 1
  XOR -> Prediksi: 0.0034 (Bulat: 0) | Target: 0
------------------------------------------------------------
```

In [6]:

```python
import matplotlib.pyplot as plt

# Membuat plot untuk memantau proses training tiap epoch dari list history manual
fig, axs = plt.subplots(3, 2, figsize=(14, 18))

# 1. Plot untuk Gerbang AND
axs[0, 0].plot(history_and_loss, color='blue', label='Loss')
axs[0, 0].set_title('Gerbang AND: Binary Cross Entropy / Loss')
axs[0, 0].set_xlabel('Epoch')
axs[0, 0].set_ylabel('Loss')
axs[0, 0].grid(True)

axs[0, 1].plot(history_and_acc, color='green', label='Accuracy')
axs[0, 1].set_title('Gerbang AND: Akurasi')
axs[0, 1].set_xlabel('Epoch')
axs[0, 1].set_ylabel('Akurasi')
axs[0, 1].grid(True)

# 2. Plot untuk Gerbang OR
axs[1, 0].plot(history_or_loss, color='blue', label='Loss')
axs[1, 1].plot(history_or_acc, color='green', label='Accuracy')
axs[1, 0].set_title('Gerbang OR: Binary Cross Entropy / Loss')
axs[1, 1].set_title('Gerbang OR: Akurasi')
axs[1, 0].set_xlabel('Epoch')
axs[1, 1].set_xlabel('Epoch')
axs[1, 0].set_ylabel('Loss')
axs[1, 1].set_ylabel('Akurasi')
axs[1, 0].grid(True)
axs[1, 1].grid(True)

# 3. Plot untuk Gerbang XOR
axs[2, 0].plot(history_xor_loss, color='blue', label='Loss')
axs[2, 1].plot(history_xor_acc, color='green', label='Accuracy')
axs[2, 0].set_title('Gerbang XOR: Binary Cross Entropy / Loss')
axs[2, 1].set_title('Gerbang XOR: Akurasi')
axs[2, 0].set_xlabel('Epoch')
axs[2, 1].set_xlabel('Epoch')
axs[2, 0].set_ylabel('Loss')
axs[2, 1].set_ylabel('Akurasi')
```

```
axs[2, 0].grid(True)
axs[2, 1].grid(True)

plt.tight_layout()
plt.show()
```











